

Q-Learning: Off-Policy Temporal-Difference Control

Learning optimal action values with epsilon-greedy experience

Learning objectives

By the end of this lesson, you should be able to:

- 01 — Derive Q-learning from the Bellman optimality target.
- 02 — Compute numeric Q updates including terminal cases.
- 03 — Implement epsilon-greedy selection with deterministic tie handling.
- 04 — Explain behavior versus target policy.
- 05 — State sufficient exploration and learning-rate convergence conditions.
- 06 — Design a defensible episodic training and evaluation loop.

Q-learning targets greedy control while behaving exploratorily

The update bootstraps from the maximum next-state action value, regardless of which next action the behavior policy takes.

- The table stores one estimate per state-action pair.
- The target uses max over next actions.
- Terminal transitions do not bootstrap.

Off-policy

Behavior gathers coverage; the target defines the greedy policy being learned.

The Q-learning update samples Bellman optimality

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

- alpha is the learning rate.
- gamma discounts continuation.
- r,s' are sampled from interaction.
- The maximum defines the greedy target.

ENGINEERING IMPLICATION

If s' is terminal, replace the maximum continuation term with zero.

Behavior and target policies have separate jobs

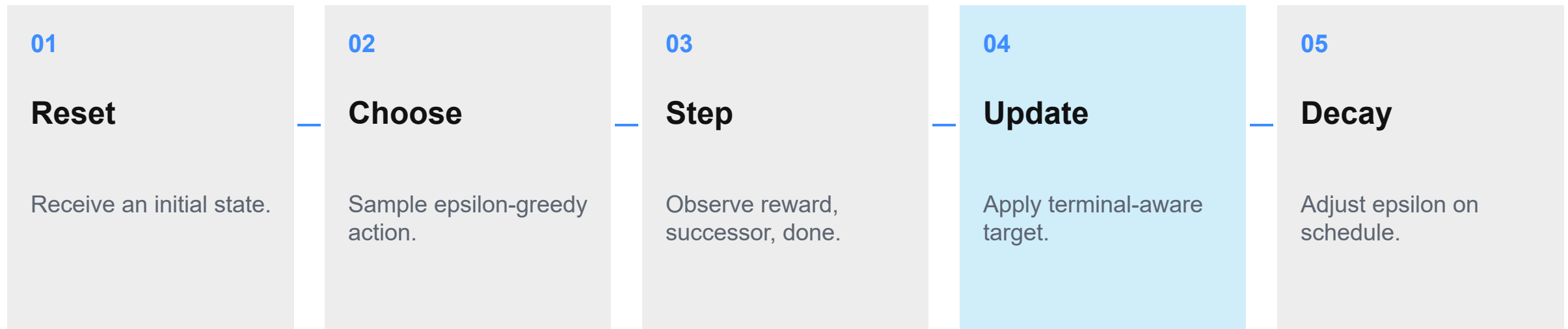
Behavior policy

- Usually epsilon-greedy.
- Generates transitions.
- Must cover relevant actions.
- Changes as Q changes.

Target policy

- Greedy with respect to Q .
- Defines bootstrap target.
- Represents learned control.
- Used without exploration at evaluation.

A Q-learning episode alternates action and update



Update exactly once per observed transition and never bootstrap past terminal.

The training loop keeps environment and agent separate

```
for episode in range(n_episodes):
    state = env.reset()
    for step in range(max_steps):
        action = epsilon_greedy(Q[state], epsilon)
        next_state, reward, done = env.step(action)
        target = reward if done else (
            reward + gamma * max(Q[next_state])
        )
        Q[state, action] += alpha * (
            target - Q[state, action]
        )
        state = next_state
        if done: break
    epsilon = decay(epsilon)
```

Defensive controls

- Validate state indexes.
- Seed tie randomness.
- Cap episode length.
- Log epsilon and return.

Worked Q update

SCENARIO

**$Q(s,a)=2$, reward=1, $\max Q(s',.)=4$,
gamma=0.9, and alpha=0.5.**

- Target= $1+0.9*4=4.6$.
- TD error= $4.6-2=2.6$.
- Increment= $0.5*2.6=1.3$.
- Set $Q(s,a)=3.3$.

Only the experienced state-action entry changes on this tabular update.

Q-learning and SARSA use different next-action targets

Q-learning

Off-policy

- Target uses $\max_a Q$.
- Learns greedy target.
- Behavior can explore.
- Can be optimistic near risk.

SARSA

On-policy

- Target uses chosen next action.
- Learns epsilon-greedy behavior value.
- Includes exploration consequences.
- Policy and data align.

Shared

TD control

- One-step bootstrap.
- Need learning rate.
- Need repeated coverage.
- Terminal target is reward.

Checkpoint: compute a terminal update

CHECKPOINT

**$Q(s,a)=5$, terminal reward=1, $\alpha=0.2$, $\gamma=0.99$.
What is the new Q value?**

01

Remove future bootstrap.

02

Compute error 1-5.

03

Apply alpha.

Epsilon-greedy must define exploration and ties

Exploration branch

- Occurs with probability ϵ .
- Sample uniformly or by documented rule.
- Can choose the current greedy action too.
- Improves state-action coverage.

Greedy branch

- Occurs with probability $1 - \epsilon$.
- Choose an argmax action.
- Tie handling changes behavior.
- Evaluation normally uses $\epsilon = 0$.

Convergence requires continued evidence and controlled steps

visit every (s,a) infinitely often; $\sum_t \alpha_t = \infty$; $\sum_t \alpha_t^2 < \infty$

- Coverage prevents permanently unknown actions.
- Nonzero total step size permits correction.
- Finite squared step size controls noise.
- Finite MDP and bounded rewards are standard assumptions.

ENGINEERING IMPLICATION

A practical finite run can only provide empirical evidence, not the asymptotic guarantee itself.

Epsilon decay changes discovery probability

SCENARIO

A sparse-reward action is initially undervalued and requires repeated trials before its benefit appears.

- High initial epsilon explores the action.
- Updates raise its Q estimate after success.
- Gradual decay shifts behavior toward exploitation.
- A too-fast decay may freeze the wrong policy.

Compare schedules across seeds and evaluate the final greedy policy separately.

Hyperparameters control distinct failure modes

Learning rate

Update size

- Too high oscillates.
- Too low learns slowly.
- Decay can stabilize.
- Log TD magnitude.

Discount

Planning horizon

- Low is myopic.
- High values delayed reward.
- Near one raises variance.
- Must match task.

Epsilon

Data coverage

- High sacrifices reward.
- Low misses alternatives.
- Decay timing matters.
- Report schedule.

Checkpoint: diagnose no learning

CHECKPOINT

A Q-table stays zero in a sparse-reward maze; epsilon decays to zero after ten episodes. What is the likely problem and first experiment?

01

Separate update logic from data coverage.

02

Inspect visited actions and rewards.

03

Change one schedule variable.

Validate Q-learning against a known-model oracle

On small environments, compare the learned greedy policy with value iteration while preserving agent-model separation.

- Unit-test one-step updates with hand values.
- Run multiple seeds and report mean plus variability.
- Evaluate without exploration or learning and check terminal behavior.

Ablation

Controlled comparisons reveal whether exploration, update logic, or evaluation causes failure.

Key takeaways

- 01 — Q-learning is off-policy TD control.
- 02 — The target is reward plus discounted maximum next Q.
- 03 — Terminal transitions never bootstrap.
- 04 — Exploration, learning rate, and evaluation protocol determine credible results.

Next: Week 6 replaces the Q-table with function approximation to generalize across large state spaces.

Check your understanding

Q1

Compute two Q-learning updates.

Q2

Contrast Q-learning with SARSA.

Q3

Explain epsilon-greedy tie handling.

Q4

State convergence conditions.

Q5

Design a sparse-reward diagnostic experiment.